



# Python for Fixed Income

COURSE 5 · ADVANCED MACHINE LEARNING

## Randomized Search and Honest Tuning

WEEK 3 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

# Week 3: the Polish rows, and today's question

## Course 4's Polish training rows

**4,680**

training rows

**326**

bankrupt (6.97%)

Every fit, score, and choice by cross-validation inside these rows.  
Course 4's 1,170 test rows stay closed.

## Today's question

**A search always finds a best candidate. Is it better, or only the luckiest?**

Primary metric: average precision, because the outcome is rare.

## Allowed names

model probability of bankruptcy in this file

flagged at threshold  $t$

## Never called

a credit view on any company

a rating

# The default model, and what a grid costs

0.7783

default LightGBM, mean of 5 folds

0.0469

fold standard deviation

100 trees, 31 leaves, learning rate 0.1, at least 20 rows per leaf

| Grid                   | Candidates | Models (5 folds) |
|------------------------|------------|------------------|
| 2 values of 3 settings | 8          | 40               |
| 5 values of 3 settings | 125        | 625              |

Cost is candidates times folds, and each setting multiplies it.

A two-value grid only ever looks at two values of each setting.

# A randomized search: fix the budget, draw the settings

```
space = {"num_leaves": randint(8, 64),  
         "learning_rate": loguniform(0.02, 0.3),  
         "min_child_samples": randint(5, 60)}
```

## **n\_iter = 6, under the grid's 8**

6 candidates: up to 6 values of every setting, where the grid tries  
2. random\_state=SEED fixes the draws.

## **The search's best**

num\_leaves 46, learning\_rate 0.1452, min\_child\_samples 25:  
best\_score\_ 0.7760

**loguniform: 0.02 to 0.03 is as likely as 0.2 to 0.3.**

# Reading the search: means inside each other's noise

| Rank | Mean   | SD     | Leaves | Rate   | Rows/leaf |
|------|--------|--------|--------|--------|-----------|
| 1    | 0.7760 | 0.0445 | 46     | 0.1452 | 25        |
| 2    | 0.7747 | 0.0564 | 22     | 0.0551 | 33        |
| 3    | 0.7742 | 0.0550 | 43     | 0.0694 | 57        |
| 4    | 0.7710 | 0.0561 | 18     | 0.0669 | 27        |
| 5    | 0.7629 | 0.0588 | 29     | 0.0294 | 7         |
| 6    | 0.7591 | 0.0622 | 51     | 0.0233 | 28        |

## search\_table

Every candidate's fold standard deviation is larger than the gap between any two neighbouring means.

**42 tests pass after today's three functions.**

# Tuning's gain against the folds' own noise

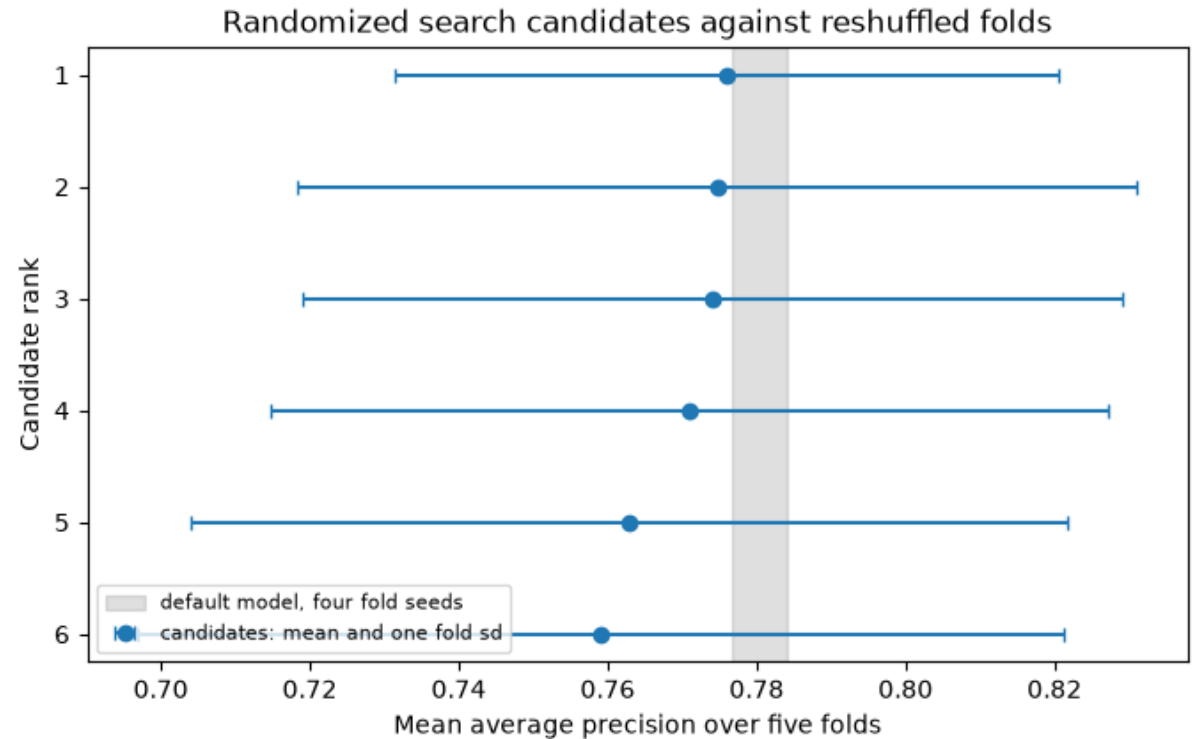
**-0.0024**

gain from tuning

**0.0075**

range, default on 4 fold seeds

| Fold seed | Default mean |
|-----------|--------------|
| 42        | 0.7783       |
| 43        | 0.7766       |
| 44        | 0.7841       |
| 45        | 0.7838       |



**Tuning moves the mean less than reshuffling the folds does.**

# Nested cross-validation: an honest score

## 1. Hold out

one outer fold

## 2. Search

on the other four, inner folds

## 3. Refit, score

on the held-out fold

## 4. Repeat

for all five

**0.7751**

nested mean

**0.7760**

best\_score\_

**0.7783**

default model

Outer folds: 0.7000, 0.8120, 0.7358, 0.7953, 0.8327; sd 0.0554

**Scored honestly, the tuned model does no better than the defaults on these rows.**

# What to tune first, and when to stop

## Defaults first

Score them: the baseline every search must beat

## Tree size next

num\_leaves or max\_depth, rows per leaf

## Then rate and trees

Together; early stopping picks the count

## Fix the budget

n\_iter and folds written before you look (n\_iter=6)

## Stop inside the noise

Gain below a fold sd or the reshuffle range

**Practice that has served well, not a rule. Log every search with its nested score.**

# The threshold is a choice too

| Model   | Threshold | Flagged | Precision |
|---------|-----------|---------|-----------|
| Default | 0.0818    | 327     | 0.7003    |
| Tuned   | 0.0181    | 336     | 0.6815    |

Recall at least 0.70 from out-of-fold probabilities, by Course 4's rule.

New settings change the scale of the outputs, so the threshold is chosen again after every search. The precision beside it is in-sample for the threshold.

Each output: the model probability of bankruptcy in this file. A flag is not a credit view.

# Excel side by side

Excel side by side (tested in Excel)

`=AVERAGE(B2:F2)`

`=STDEV.P(B2:F2)`

`=STDEV.S(B2:F2)`

`=MAX(G2:G7)`

`=AVERAGE(A20:E20)`

`=MAX(A24:A27)-MIN(A24:A27)`

a candidate's mean, filled down

std\_test\_score: 0.0445 for the best

larger: 0.0497 (divides by 4)

best\_score\_ 0.7760

nested mean 0.7751

fold-seed range 0.0075

**No randomized search and no cross-validation in Excel. It reads a search pasted onto a sheet.**

# Save the day: reading a commit

```
git show --stat HEAD
```

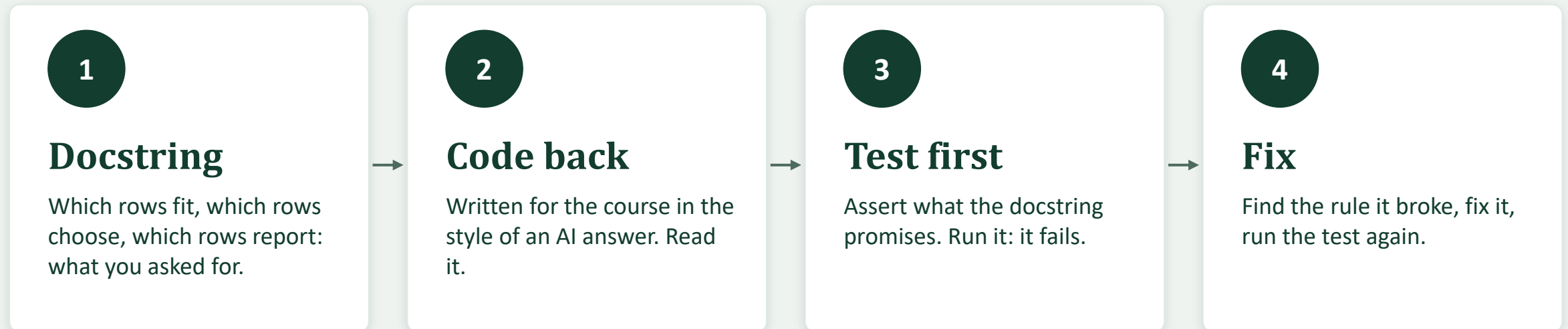
The last commit's message, then one line per file: its name, lines changed, and a bar of + for lines added.

```
model,cv_mean,cv_sd,nested_mean  
lightgbm_default,0.7783,0.0469,0.7783  
lightgbm_random_search,0.7760,0.0445,0.7751
```

A tuning day's log: `best_score_` and the nested mean, side by side.

# The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.



Today: `best_score_` logged as the tuned model's honest score; the test asks for the nested mean.

*Code that runs is not the same as code that is right.*

# What you can do now

- Rebuild Course 4's Polish rows; keep its test rows closed
- A grid costs candidates times folds
- A randomized search covers more values for the same budget
- Measure a gain against reshuffled folds
- Report the nested score beside `best_score_`

**Week 3, Day 2** Feature engineering on financial ratios